正则化：数据增强、参数共享与权重衰减

读前须知

- **前置**：过拟合与偏差—方差；知道 L1、L2 范数的含义。
- **这一篇的位置**：深度模型能不能用的分水岭。网络容量足够大时，正则化决定了它学的是规律还是噪声。
- **读完能做什么**：说清权重衰减与 L2 正则的等价条件；解释 Dropout 为什么相当于集成；为图像、文本任务各选一套数据增强策略。

一、正则化要解决什么

1.1 泛化误差与过拟合

正则化（Regularization）作为抑制过拟合的手段，是机器学习和深度学习之中必不可少的环节，具有举足轻重的地位。

好的机器学习算法不仅要在训练集上表现出色，当推广到未知的测试数据时，其优良的性能依然能够得以保持。正则化就是一类通过显式设计降低泛化误差，以提升算法通用性的策略的统称。由于深度学习中涉及的参数众多，正则化就变得尤为重要。

参数多意味着假设空间大，假设空间大意味着模型有足够的能力把训练数据中的噪声也一并记下来。这是过拟合的根源。深度网络的参数量动辄百万千万，而训练样本可能只有几万，从“参数多于样本”这个角度看，过拟合几乎是默认状态，所以正则化不是可选项。

1.2 奥卡姆剃刀

正则化被定义为对学习算法的修改，这些修改的目的在于减少泛化误差。通常说来，泛化误差的下降是以训练误差的上升为代价的，但有些算法也能兼顾泛化误差和训练误差的良好性能。

正则化处理可以看成是奥卡姆剃刀原则（Occam's razor）在学习算法上的应用。奥卡姆剃刀原则的表述是：“当两个假说具有完全相同的解释力和预测力时，以那个较为简单的假说作为讨论依据。”在机器学习中，正则化处理得到的正是更加简单的模型。

从概率论角度看，许多正则化技术对应的是在模型参数上施加一定的先验分布，其作用是改变泛化误差的结构。正则化是对欠拟合和过拟合的折中，在不过度增加偏差的情况下显著减少方差。正则化能够改变数据分布，让通过模型得到的数据分布尽可能和真实的数据生成过程相匹配。

> **新手提示：正则化不是“把模型变小”**
>
> 一个常见的误解是正则化等于降低模型容量，实际上它改变的是“在同样容量下偏好哪一类解”。以 L2 正则化为例，模型的假设空间没有变小——任意权重组合依然可以取到，只是损失函数被修改，使得权重较大的解代价更高。从优化结果看，模型倾向于落在权值较小的区域，表现为“更简单”，但这是目标函数改变后的结果，不是架构被裁剪。理解这一点，才能明白为什么正则化强度需要调节：太弱不管用，太强会把有用的权重也压掉。

二、正则化的整体框架

2.1 学习算法的流程

虽然目前在深度学习中应用的正则化方式称得上“八仙过海，各显神通”，却并不存在能够系统描述这些方法、并进一步指导设计的通用主线。因此，要从通观全局的角度看待正则化处理，还是要“不忘初心”，从根本目的着眼。

机器学习的任务是拟合出一个从输入 x 到输出 y 的分布，拟合的过程是使期望风险函数最小化的过程。正则化处理使待最小化的函数中既包含结构化的误差函数，也包含人为引入的正则化项。

由于未知分布的期望风险不能直接求解，因而需要引入训练数据集，以在训练数据集上计算出的经验风险来近似期望风险，并通过经验风险最小化实现期望风险最小化。以上就是学习算法的整体流程，也是正则化大展拳脚的主战场。

2.2 五类变量的划分

正则化的处理就是针对学习算法中的不同变量来展开的。这样看来，正则化策略就可以分为以下几类：

1. **基于训练数据**（data）的正则化：改变送入网络的数据本身；
2. **基于网络架构**（network architecture）的正则化：改变模型的连接方式与假设空间；
3. **基于误差函数**（error function）的正则化：改变优化的目标；
4. **基于正则化项**（the regularization term）的正则化：在目标上加惩罚；
5. **基于最优化过程**（optimization）的正则化：改变求解路径。

这个划分的价值在于它回答了“从哪里下手”。数据、架构、目标函数、惩罚项、求解过程，正好是学习算法的五个环节，每一个环节都可以插入约束。下面按这个顺序展开。

三、基于训练数据

3.1 数据增强

训练模型的质量很大程度上取决于训练数据。除了选择噪声较小的训练数据外，还可以通过正则化来提升训练数据的质量。

正则化处理数据的一个目的是执行预处理和特征提取，从而将特征空间或数据分布修改为其他形式；另一个目的是通过生成新样本来创建具有更大容量、甚至是无限容量的增强数据集。这两个目的之间相互独立，因而可以结合起来使用。

对训练数据正则化的做法是在训练数据集上施加变换，从而产生新的训练数据集。变换的形式是以满足某种概率分布的随机变量为自变量的函数，最简单的实例就是向数据添加随机的高斯噪声。由于提升机器学习模型泛化性能最直接的办法就是使用更多的数据进行训练，因而使用随机参数的变换可以用于生成“假”数据，这种方法被称为**数据集增强**（data augmentation）。

数据集增强的对象通常只包括输入层和隐藏层，而不包括输出层。这样做的效果是将训练数据集映射到一个新的概率分布上，并用这个新分布来计算经验风险函数。变换中参数的随机性使我们可以在不同的参数之下生成多组新数据，从而通过数据量的增加来减小期望风险和经验风险之间的差别。

补充说明：数据增强的关键是“变换必须保持标签不变”。图像水平翻转后猫依然是猫，所以翻转是合法的增强；但把数字 6 上下翻转就变成了 9，这种变换会破坏标签，不能用。按这个原则，图像常用的增强包括随机裁剪、旋转、色彩抖动、Mixup（把两张图按比例混合、标签也按同样比例混合）与 CutMix（把一张图的局部贴到另一张上）。语音常用加噪、变速、变调；文本常用同义替换、回译。增强策略设计得越贴近真实世界的变化，泛化收益越大。

3.2 Dropout

除了数据集增强外，另一种针对训练数据的正则化方法是 **Dropout**。Dropout 是一种集成方法，通过结合多个模型来降低泛化误差。

之所以说 Dropout 属于基于训练数据的正则化，是因为它构造不同的数据集来训练不同的模型，每个数据集则通过对原始数据集进行“有放回采样”得到。Dropout 的关键想法是在训练期间从神经网络中随机丢弃神经元及其连接，得到简化的网络。而在测试时，一个简单的小权重网络就可以逼近所有这些简化网络的预测的平均效果。其优点在于计算简单方便，同时还具有对不同的模型和训练过程的普适性。但 Dropout 对训练集容量的要求很高，少量训练样本并不能发挥其优势。

补充说明：标准 Dropout 有一个实现上的坑。若训练时以概率 p 保留神经元，则训练阶段该神经元的期望输出被缩放到原来的 p 倍；而推理时若不做处理，激活值会比训练时大 1/p 倍，导致预测偏移。早期做法是在推理时把权重乘以 p，叫“非倒置 Dropout”。现在框架默认采用**倒置 Dropout**：训练时就把保留的激活值除以 p，推理时不做任何缩放，直接使用全部神经元。这样模型在训练与推理时的期望输出一致，也便于在推理阶段关闭 Dropout 而不改代码。

> **新手提示：推理前记得切换 eval 模式**
>
> 用 PyTorch 这类框架搭网络时，Dropout 与批归一化在训练和推理时的行为不同。若在推理时忘了调用 model.eval()，Dropout 会继续随机屏蔽神经元，同一个输入两次前向传播得到的结果不一样，模型表现会莫名其妙地变差，而且这种错误极难从结果上定位。另一个容易混淆的点是：model.eval() 只改变层的行为，不会关掉梯度计算；要省显存还得配合 torch.no_grad()。验证与推理两个开关要分清楚。

3.3 Dropout 作为集成

把 Dropout 看成集成，可以解释它为什么有效。

一个含有 n 个神经元的网络，在 Dropout 下可以产生 2^n 种不同的子网络。每次迭代只训练其中一个子网络，但所有子网络共享权重。训练结束后，权重相当于在所有子网络上做了一次参数平均，而推理时使用的完整网络可以看作这 2^n 个子网络的几何平均近似。集成的方差低于单个模型，所以泛化误差下降。

这个视角也解释了 Dropout 的局限：子网络数量虽多，但它们共享权重，多样性远不如训练多个独立模型；而且当训练样本很少时，每次迭代落到的子网络各不相同，模型很难收敛到一致的表征。

四、基于网络架构

4.1 参数共享

从输入到输出的映射必须具有某些特质才能很好地适应数据，而对输入—输出映射进行假设的方法正对应着网络结构的选择，这激发了基于网络架构的正则化方法。对映射的假设既可以关注深度网络中不同层次的具体操作，也可以关注层与层之间的连接方式。

基于网络架构的正则化通常会简化关于映射的假设，再让网络架构逐步逼近简化后的映射。这限制了模型的搜索空间，为找到更好的解提供了可能性。

参数共享（weight sharing）是一类重用参数的正则化方法。通过强迫某些参数相等，可以让不同模型共享唯一的参数，从而让它们对相似的输入产生相似的输出。如果放宽参数共享的条件，使它们不必完全相等而是相互接近，对应的就是对参数范数添加正则化项。常用的参数共享方法是将一个监督学习模型的参数正则化，令其接近另一个无监督学习的模型，那么这个无监督学习模型就可以匹配监督模型的参数。

补充说明：参数共享最成功的例子就是卷积。一个 3×3 卷积核在整幅图像的所有位置上重复使用，等于强行假设"同一个特征在任何位置的含义都相同"，这就是平移等变性。这个假设把参数量从"与图像尺寸的平方成正比"降到"只与核大小和通道数有关"，降幅可达几个数量级。循环网络是另一个例子：同一个循环单元在时间步之间共享，等于假设"不同时刻的动力学一致"。嵌入层共享、多任务模型共享底层，也都是同一思路——把"这些位置应当行为一致"的先验直接写进结构。

4.2 传递函数正则化

另一种针对网络架构的处理是对传递函数的正则化（activation regularization）。一些传递函数是专门为正则化设计的，比如在 Dropout 中使用的 maxout 单元，它能在测试时更精确地近似模型集合预测结果的几何平均值。而通过添加噪声，原始的确定传递函数就可以被泛化为随机模型，其分布特性也就可以被利用起来。

五、基于误差函数与正则化项

5.1 惩罚项的含义

基于误差函数的正则化和基于正则化项的正则化可以放在一起讨论。理想情况下，误差函数应当适当地反映算法的性能，并体现出数据分布的一些特点（比如均方误差或交叉熵）。对误差函数进行正则化就相当于添加额外的学习任务，从而导致其目标发生变化，这部分变化就会体现在误差函数中额外的正则化项上。因而在大部分情况下，对基于正则化项的正则化的讨论就包含了基于误差函数的正则化。

正则化项也叫做**惩罚项**。与误差函数不同，正则化项与目标无关，而是用于表示所需模型的其他属性。误差函数表示的是算法输出与目标输出之间的一致性，正则化项表示的则是关于映射关系的额外的假设。这一特点决定了正则化项的值可以通过未标记的测试样本来计算，利用测试数据改进学习模型。

合起来写，待最小化的目标变成：

$$\tilde{J}(\theta) = J(\theta) + \alpha\,\Omega(\theta)$$

其中 J 是原来的误差项，Ω 是正则化项，α 控制两者的相对权重。α 为零时退化为普通训练，α 过大时模型被压得过于简单，会出现欠拟合。

5.2 权重衰减

常用的正则化项是**权重衰减项**（weight decay）。深度学习中的参数包括每个神经元中的权重系数和偏置。由于每个权重会指定两个变量之间相互作用的方式，因而拟合权重所需要的数据量要比拟合偏置多得多。相比之下，每个偏置只控制一个变量，即使不对它做正则化也不会产生太大方差，正则化的方式不对反而还会增加算法的偏差。这是正则化的对象只包括权重而不包括偏置的原因。

在权重衰减中，正则化项是以范数的形式表示的，常用的范数包括 L2 范数和 L1 范数。这两种范数作为正则化项应用在线性回归中时，分别对应着岭回归和 LASSO 回归。

L2 范数作为正则化项时，其作用是使权重系数更加接近原点：

$$\Omega(\theta) = \frac{1}{2}\|\theta\|_2^2$$

引入权重衰减后，在每一步的梯度更新之前，权重向量都会被收缩：

$$\theta \leftarrow \theta - \eta\left(\nabla_\theta J(\theta) + \alpha\theta\right) = (1-\eta\alpha)\,\theta - \eta\nabla_\theta J(\theta)$$

乘上 (1−ηα) 这一步就是"衰减"一词的由来。整体来看，这使得在显著减小目标函数方向上的权重保留完好，无益于目标函数减小的方向所对应的分量则会因正则化而被逐渐地衰减掉。从泛化误差的角度来说，L2 范数能够感知具有较高方差的输入，与这些输入特征相关的权重则被收缩。

补充说明：L2 正则化等价于对权重施加高斯先验。把负对数后验展开：

$$-\ln p(\theta \mid D) \propto -\ln p(D \mid \theta) - \ln p(\theta)$$

若先验 p(θ) 是均值为零的高斯分布，则 −ln p(θ) 正比于 $\|\theta\|^2$，与 L2 项形式完全一致。同理，若先验是拉普拉斯分布，负对数先验正比于 $\|\theta\|_1$，得到的就是 L1 正则化。所以"加惩罚项"和"加先验"是同一件事的两种说法：惩罚项越强，对应的先验越确信参数应当接近零。这也解释了为什么正则化强度 α 可以直接理解为先验的精度。

5.3 L1 与 L2 的区别

相比之下，L1 范数和 L2 范数有本质上的区别。L1 正则化得到的是稀疏的解，它将一部分较小的权重直接砍掉：

$$\Omega(\theta) = \|\theta\|_1 = \sum_i |\theta_i|$$

这样做可以从可用的特征子集中选择出有意义的特征，从而简化学习问题。

两者差异的根源在几何形状。L2 的等值面是球面，与损失函数等值面相交时，交点通常落在某个坐标不为零的位置；L1 的等值面是菱形，顶点恰好落在坐标轴上，交点更容易落在某个坐标为零的顶点上，于是产生稀疏解。换言之，L2 把所有分量都往零压，但压不到零；L1 会把一部分分量压成精确的零。

六、基于最优化过程

6.1 初始化与参数更新

最后一类正则化方法是基于最优化过程的正则化。根据其作用阶段的不同，这类正则化方法可以分为三种：对初始化（initialization）的正则化，对参数更新（weight update）的正则化，对终止条件（termination）的正则化。

对初始化的正则化影响的是权重系数的最初选择，这既可以通过在特定的概率分布中选择初始参数完成，也可以经由预训练实现。对参数更新的正则化则包括对更新规则的改进和对权重参数的过滤。

6.2 早停

早停（early stopping）是一类重要的正则化方法，它是针对终止条件的正则化。

当训练的表示能力过强时，泛化误差会呈现出 U 形：随着训练时间的增加先降低再升高。这意味着只有返回使泛化误差最低的参数集设置，才能得到更低的泛化误差。当测试集上的泛化误差在预先指定的循环次数内没有改善时，训练就会终止，这种策略就是早停。

早停的简单性和有效性使它成为深度学习中应用最广泛的正则化方法。它不需要改变训练过程，也就不会影响到学习动态，唯一要注意的问题就是要避免陷入局部最优解。

> **新手提示：早停用的不是测试集**
>
> 教科书常写"当测试集误差不再下降时停止"，这在实践中是错的——测试集一旦参与训练决策，就不再是测试集，最终报告的性能会偏乐观。正确做法是从训练集里切出一份**验证集**，用验证集监控早停，测试集只在最后评估一次。若数据量很小，可以用 k 折交叉验证代替固定验证集，或者用"训练损失在一段时间内不再下降"作为近似判据。监控指标要与最终目标一致：分类任务看准确率或 F1，而不是训练损失。

补充说明：早停与 L2 正则化的等价性可以直观理解。把参数从初始化 θ_0 出发的轨迹按步数展开，早停在 t 步停下的解，其偏离初始化的距离约为每步更新量的累加，与 L2 正则化把解约束在以 θ_0 为球心的球内是同一类约束。在二次损失与线性化假设下，两者可以证明给出相同的解。这个等价性的实践意义是：早停的轮数可以当作正则化强度来调——轮数越少，约束越强，越接近强 L2。

七、几类方法的配合

7.1 方法不互斥

需要注意的是，以上的分类方式并不是互斥的，一种正则化方法通常针对的是学习算法中的多个变量。做出分类的目的是帮助更好地理解正则化的设计思路。此外，对具体正则化方法的介绍也是挂一漏万。正则化是深度学习研究中最活跃的领域之一，如果要深入了解每种方法，最好的办法莫过于阅读原始文献。

以 Dropout 为例，它既改变训练数据（每次的子网络等价于在随机采样的数据上训练），又改变网络架构（随机屏蔽连接），还隐含了对权重的约束（相当于 L2 的一种近似）。分类是为了看清它作用于哪个环节，而不是给它贴一个唯一标签。

7.2 如何选择

实际工程中的顺序通常是：先上数据增强与早停，这两者几乎无成本且收益稳定；再考虑权重衰减，它的强度需要与学习率一起调；然后才考虑 Dropout 一类结构性方法，因为它们会改变训练动态。架构层面的正则化（参数共享）通常在设计网络时就已确定，不是事后可调的开关。

补充说明：现代实践中有两个容易踩的细节。其一是 **Adam 与 L2 的耦合问题**：在 Adam 中直接把 L2 项加进梯度，会被自适应学习率按参数尺度缩放，导致大权重的实际衰减量被削弱，效果不同于真正的权重衰减。AdamW 把衰减项从梯度中解耦，直接作用在参数上，这是目前训练 Transformer 的标准做法。其二是**双下降**：在过参数化区间，增强正则化未必降低测试误差，反而可能因为破坏了插值解而变差。正则化强度不能凭直觉一路加大，必须用验证集实测。

四个常见误区

- **“Dropout 训练和推理时一样用”**——推理时必须关闭 Dropout，并做相应缩放，否则输出期望会偏移。现代实现多在训练时放大，推理时直接关闭。
- **“权重衰减就等于 L2 正则”**——在普通 SGD 下近似等价，在 Adam 这类自适应优化器下并不等价，这也是 AdamW 把权重衰减解耦出来的原因。
- **“正则化越强越好”**——过强的正则会让模型欠拟合，训练误差与验证误差同时变差。
- **“数据增强不会改变分布”**——翻转、裁剪、颜色扰动都在改写数据分布。增强方式与目标任务不匹配时，会引入偏差而不是鲁棒性。

八、要点回顾

1. 正则化通过修改学习算法降低泛化误差，通常以训练误差上升为代价。
2. 正则化可视为奥卡姆剃刀的应用，在概率视角下相当于对参数施加先验分布。
3. 按作用环节可分为五类：训练数据、网络架构、误差函数、正则化项、最优化过程。
4. 数据增强通过对输入施加保标签的随机变换扩充数据集，减小经验风险与期望风险的差距。
5. Dropout 训练时随机丢弃神经元，推理时使用完整网络，可看作大量子网络的集成。
6. 参数共享通过强迫参数相等限制假设空间，卷积与循环网络的参数复用是典型例子。
7. 正则化项的通用形式为 $J(\theta) + \alpha\Omega(\theta)$，$\alpha$ 控制约束强度。
8. 权重衰减只作用于权重不作用于偏置，因为偏置的方差小、正则化反而增加偏差。
9. L2 使权重整体收缩，对应高斯先验；L1 产生稀疏解，对应拉普拉斯先验。
10. 早停针对终止条件，用验证集监控，在一定意义下与 L2 正则化等价。
11. 各类方法并不互斥，实践中通常先上数据增强与早停，再调权重衰减与结构性方法。

九、名词汇总

名称	英文全称	英文缩写	简短介绍
正则化	Regularization	—	通过修改学习算法以降低泛化误差的一类策略
过拟合	Overfitting	—	模型记住训练数据噪声、在新数据上表现变差的现象
欠拟合	Underfitting	—	模型容量不足、训练误差本身就很高的状态
泛化误差	Generalization Error	—	模型在未见数据上的期望误差
经验风险	Empirical Risk	—	在训练集上计算出的平均损失
期望风险	Expected Risk	—	在真实数据分布上的期望损失
奥卡姆剃刀	Occam's Razor	—	解释力相同时优先选择更简单假设的原则
偏差方差折中	Bias-Variance Tradeoff	—	模型复杂度在欠拟合与过拟合之间的平衡
数据增强	Data Augmentation	—	对训练样本施加保持标签的随机变换以扩充数据
Dropout	Dropout	—	训练时随机屏蔽神经元、推理时使用全网络的集成式正则化
倒置 Dropout	Inverted Dropout	—	训练时按保留概率缩放激活、推理时不缩放的做法
参数共享	Weight Sharing	—	让多个位置或模块使用同一组参数的结构约束
平移等变性	Translation Equivariance	—	输入平移时输出随之平移的性质，卷积核共享的来源
权重衰减	Weight Decay	—	在参数更新中按比例收缩权重的正则化方法
L1 正则化	L1 Regularization	—	以权重绝对值之和为惩罚项，产生稀疏解
L2 正则化	L2 Regularization	—	以权重平方和为惩罚项，使权重整体收缩
岭回归	Ridge Regression	—	带 L2 正则化的线性回归
LASSO 回归	Least Absolute Shrinkage and Selection Operator	LASSO	带 L1 正则化的线性回归，具备特征选择能力

名称	英文全称	英文缩写	简短介绍
拉普拉斯先验	Laplace Prior	—	对应 L1 正则化的参数先验分布
高斯先验	Gaussian Prior	—	对应 L2 正则化的参数先验分布
早停	Early Stopping	—	验证集指标不再改善时终止训练的正则化手段
验证集	Validation Set	—	用于调参与早停、不参与最终评估的数据子集
交叉验证	Cross Validation	CV	轮流留出一部分数据作验证的评估方法
AdamW	Adam with Decoupled Weight Decay	AdamW	将权重衰减与梯度更新解耦的 Adam 变体
双下降	Double Descent	—	测试误差随模型复杂度先升后降的现象